

Boosting and bagging models for (1) Continuous, (2) Binary and (3) Multicategory responses: using the students performance data

Ikedichi Edward Azuh

4/12/2021

Students performance data

This dataset contains the examination scores of 1,000 students in three subjects: mathematics, reading, and writing. In addition to the test scores, it includes several demographic and socioeconomic variables, such as gender, race/ethnicity, parental level of education, lunch type, and participation in a test preparation course. The primary aim of the dataset is to investigate how factors such as family background, parental education, and test preparation influence student academic performance.

In this lecture, we use the dataset to illustrate boosting and bagging methods for three different types of response variables:

1. **Continuous response:** We define the response variable as the student's **average score**, computed as the mean of the mathematics, reading, and writing scores.
2. **Binary response:** We create a pass/fail outcome based on the average score, where students with scores from 1–49 are classified as **Fail** and those with scores from 50–100 are classified as **Pass**.
3. **Multicategory response:** We further categorise the average score into three performance levels:
 - **Poor:** average score < 55 ,
 - **Good:** average score between 55 and 80,
 - **Excellent:** average score > 80 .

Remark: Each of these transformations leads to a different statistical learning problem:

- Continuous response $\rightarrow$ regression problem
- Binary response $\rightarrow$ classification with two classes
- Multiclass response $\rightarrow$ multinomial classification

Although the response changes, we keep the same predictor set throughout to ensure comparability across models.

These three response variables allow us to demonstrate how ensemble learning techniques such as bagging and boosting can be adapted to regression, binary classification, and multiclass classification problems within a single, coherent case study.

The dataset was obtained from the Kaggle repository and is used here solely for educational and teaching purposes. For access to the data and further details, see <https://www.kaggle.com/spscientist/students-performance-in-exams>.

Load data

```
Student_perf <- read.csv("StudentsPerformance.csv")

Student_perf$gender <- as.factor(Student_perf$gender)
Student_perf$race.ethnicity <- as.factor(Student_perf$race.ethnicity)
Student_perf$parental.level.of.education <- as.factor(Student_perf$parental.level.of.education)
Student_perf$lunch <- as.factor(Student_perf$lunch)
Student_perf$test.preparation.course <- as.factor(Student_perf$test.preparation.course)
Student_perf$Ave_score <- (Student_perf$math.score + Student_perf$reading.score +
                          Student_perf$writing.score)/3

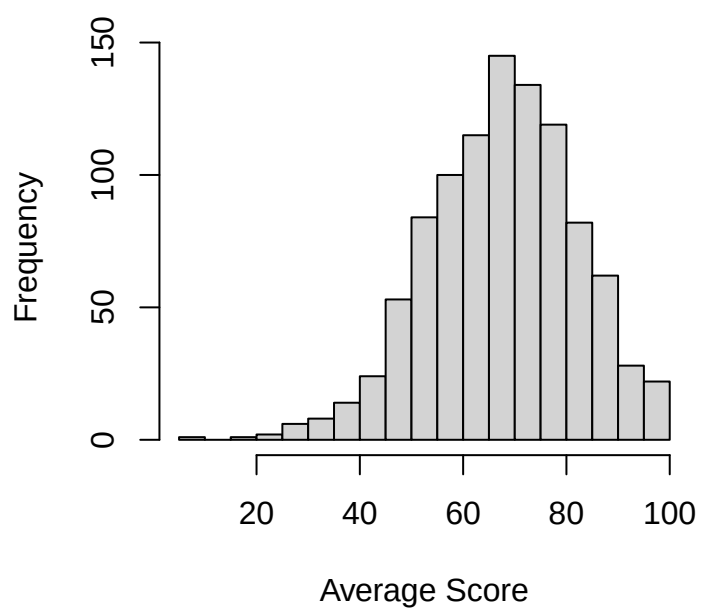
str(Student_perf)

## 'data.frame':    1000 obs. of  9 variables:
## $ gender          : Factor w/ 2 levels "female","male": 1 1 1 2 2 1 1 2 2 1 ...
## $ race.ethnicity   : Factor w/ 5 levels "group A","group B",...: 2 3 2 1 3 2 2 2 4 2 ...
## $ parental.level.of.education: Factor w/ 6 levels "associate's degree",...: 2 5 4 1 5 1 5 5 3 3 ...
## $ lunch            : Factor w/ 2 levels "free/reduced",...: 2 2 2 1 2 2 2 1 1 1 ...
## $ test.preparation.course : Factor w/ 2 levels "completed","none": 2 1 2 2 2 2 1 2 1 2 ...
## $ math.score        : int  72 69 90 47 76 71 88 40 64 38 ...
## $ reading.score      : int  72 90 95 57 78 83 95 43 64 60 ...
## $ writing.score       : int  74 88 93 44 75 78 92 39 67 50 ...
## $ Ave_score          : num  72.7 82.3 92.7 49.3 76.3 ...
```

Visualization

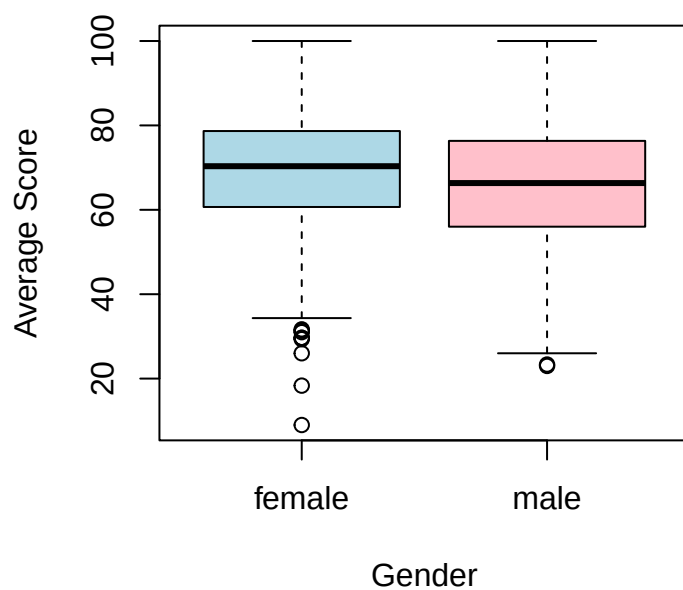
```
hist(Student_perf$Ave_score, breaks = 20, main = "Distribution of Average Score",
      xlab = "Average Score")
```

Distribution of Average Score

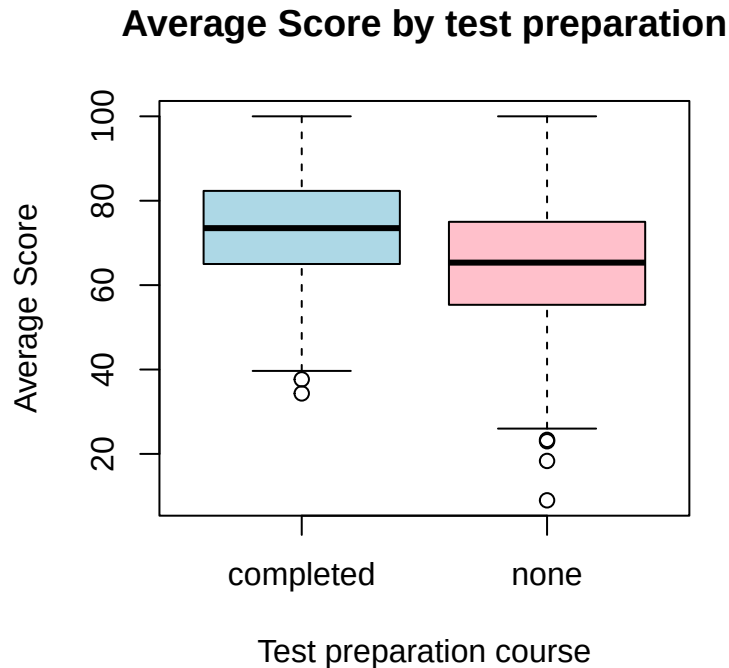


```
boxplot(Ave_score ~ gender, data = Student_perf, col = c("lightblue","pink"),  
        main = "Average Score by gender", xlab = "Gender", ylab = "Average Score")
```

Average Score by gender



```
boxplot(Ave_score ~ test.preparation.course, data = Student_perf, col=c("lightblue", "pink"),
        main = "Average Score by test preparation", xlab = "Test preparation course",
        ylab = "Average Score")
```



The distribution of average scores is approximately symmetric, with most students scoring between 55 and 80. Preliminary plots also suggest differences in performance across gender and test preparation status.

The boxplots suggest that students who completed the test preparation course tend to achieve higher average scores. These preliminary findings motivate the use of predictive models to quantify the relationship between student characteristics and academic performance.

Models (for continuous response variable)

1.0 Gradient Boosting for linear models using glmboost()

We use component-wise gradient boosting, a stagewise estimation procedure in which simple base-learners (typically linear effects) are fitted sequentially to the negative gradient of a specified loss function. At each iteration, the algorithm updates the model by selecting the base-learner that produces the greatest reduction in the empirical risk (prediction error).

Depending on the chosen loss function (e.g., **L2**, **L1**, or **Huber**), the same framework can be used for least-squares regression, median regression, robust regression, and many other models. These loss functions determine how prediction errors are penalized during optimization. The L2 loss strongly penalizes large deviations and is therefore sensitive to outliers, the L1 loss is more robust to outliers, while the Huber loss provides a compromise between the two by combining quadratic and absolute loss behaviour.

Remark: In component-wise boosting, each iteration updates only one base-learner at a time. Consequently, boosting can also be viewed as a form of variable selection, since predictors that do not improve the loss function are selected less frequently or may not be selected at all.

```
library(caTools)

# Data partitioning into Train and Test

set.seed(3000)
spl = sample.split(Student_perf$Ave_score, SplitRatio = 0.7)
Train = subset(Student_perf, spl==TRUE)
Test = subset(Student_perf, spl==FALSE)
```

1.1 L2 regression (Standard regression)

The L2 boosting minimizes the least square error (sum of squares of the difference between the observed and the estimated values). It is very sensitive to outliers.

```
library(mboost)
fm = Ave_score ~ gender + test.preparation.course + lunch + race.ethnicity +
      parental.level.of.education

time_fit_1 <- system.time(fit1 <- glmboost(fm, data=Train, control = boost_control(mstop = 1000),
      family = Gaussian(), center = FALSE)) #default mstop is 100

time_1 <- round(as.numeric(time_fit_1["elapsed"]),2)

# Predictions:
pred1 <- predict(fit1, newdata=Test, type="response")

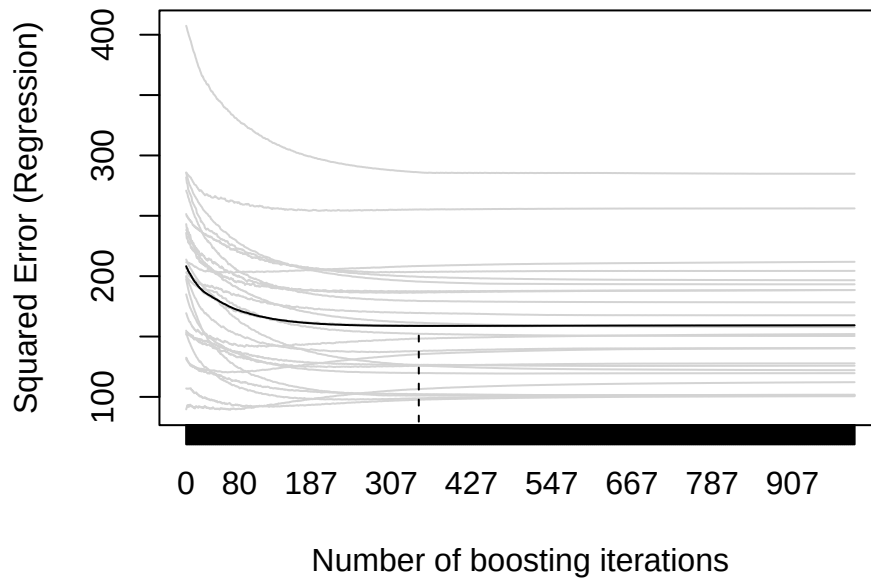
Acc1 <- round(cor(pred1,Test[,9])^2*100, 2) # Accuracy is 19.68%

library(Metrics)
MSE1 <- round(mse(Test[,9], pred1),2)
RMSE1 <- round(sqrt(mean((Test[,9] - pred1)^2)),2)
MAE1 <- round(mae(Test[,9], pred1),2)

set.seed(123)
fit1.cv <- cvrisk(fit1, folds = cv(model.weights(fit1), type = "kfold", B=25))
# B=no of resampling splits (folds)

plot(fit1.cv) # vertical dashed line show the iteration where error is minimum
```

25-fold kfold



1.2 L1 regression (Median regression)

The L1 boosting minimizes the least absolute deviations (sum of the absolute difference between the observed and the estimated values). It is more robust than L2.

```
time_fit2 <- system.time(fit2 <- glmboost(fm, data=Train, control = boost_control(mstop = 1000),
                                         family = Laplace(), center = FALSE))

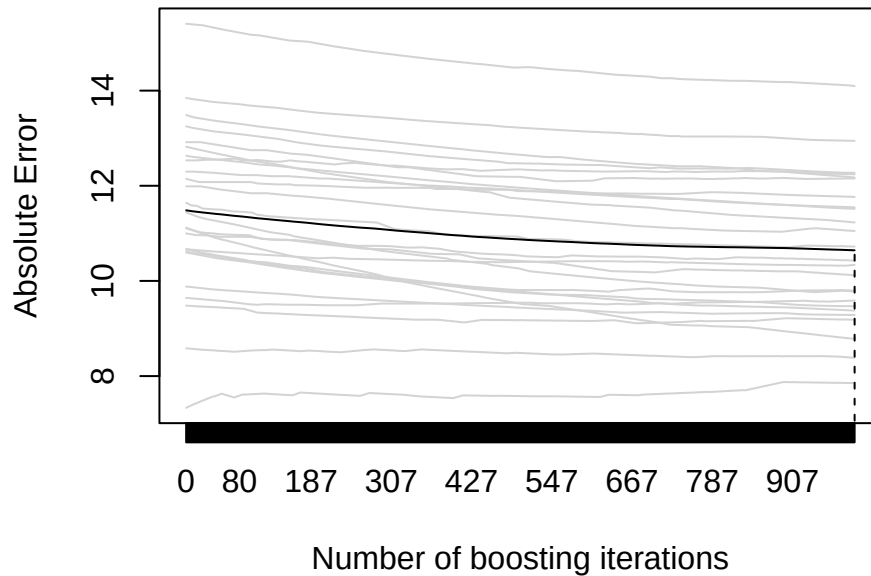
time_2 <- round(as.numeric(time_fit2["elapsed"]), 2)

# Predictions:
pred2 <- predict(fit2, newdata=Test, type="response")

Acc2 <- round(cor(pred2, Test[,9])^2*100, 2) # Accuracy is 15.22%
MSE2 <- round(mse(Test[,9], pred2), 2)
RMSE2 <- round(sqrt(mean((Test[,9] - pred2)^2)), 2)
MAE2 <- round(mae(Test[,9], pred2), 2)

set.seed(123)
fit2.cv <- cvrisk(fit2, folds = cv(model.weights(fit2), type = "kfold", B=25)) # B=no of folds
plot(fit2.cv)
```

25-fold kfold



```
#mstop(fit2.cv)
```

1.3 Robust regression

Uses the Huber loss function (less sensitive to outliers than the L2 loss function).

```
time_fit3 <- system.time(fit3 <- glmboost(fm, data=Train, control=boost_control(mstop = 1000),
                                     family = Huber(), center = FALSE))

time_3 <- round(as.numeric(time_fit3["elapsed"]),2)

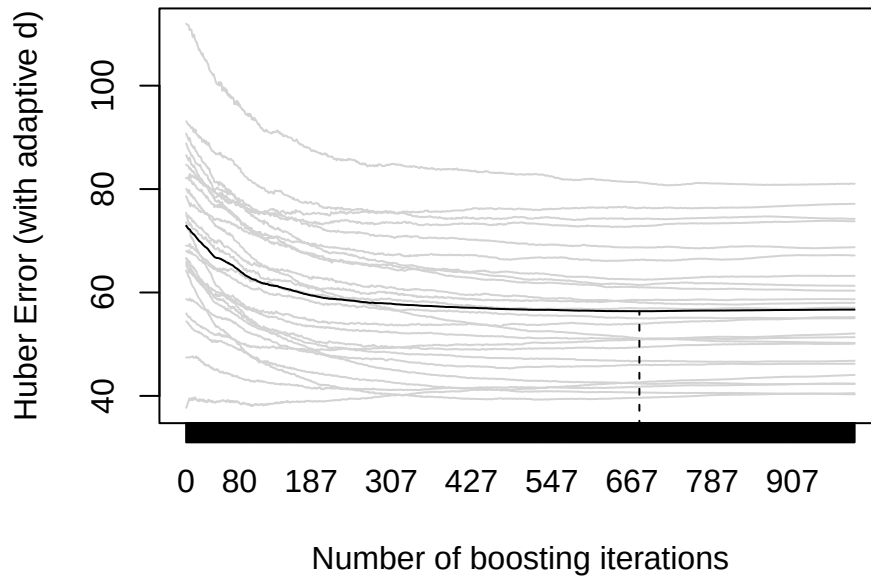
# Predictions:
pred3 <- predict(fit3, newdata=Test, type="response")

Acc3 <- round(cor(pred3,Test[,9])^2*100, 2) # Accuracy is 19.24%
MSE3 <- round(mse(Test[,9], pred3) ,2)
RMSE3 <- round(sqrt(mean((Test[,9] - pred3)^2)), 2)
MAE3 <- round(mae(Test[,9], pred3) ,2)

set.seed(123)
fit3.cv <- cvrisk(fit3, folds = cv(model.weights(fit3), type = "kfold", B=25)) # B=no of folds

plot(fit3.cv)
```

25-fold kfold



```
#mstop(fit3.cv)
```

1.4 Quantile regression

Models the relationship between the independent variables and the conditional quantiles of the response variable. The family here is *QuantReg*. It is important to note that fitting quantile regression generally requires higher stopping iterations than standard regression with the L2 loss, as the negative gradients which are fitted to the base-learners are vectors containing only small values.

```
time_fit4 <- system.time(fit4 <- glmboost(fm, data=Train, control = boost_control(mstop = 1000),
                                         family = QuantReg(tau = 0.5, qoffset = 0.5), center = FALSE))

time_4 <- round(as.numeric(time_fit4["elapsed"]), 2)

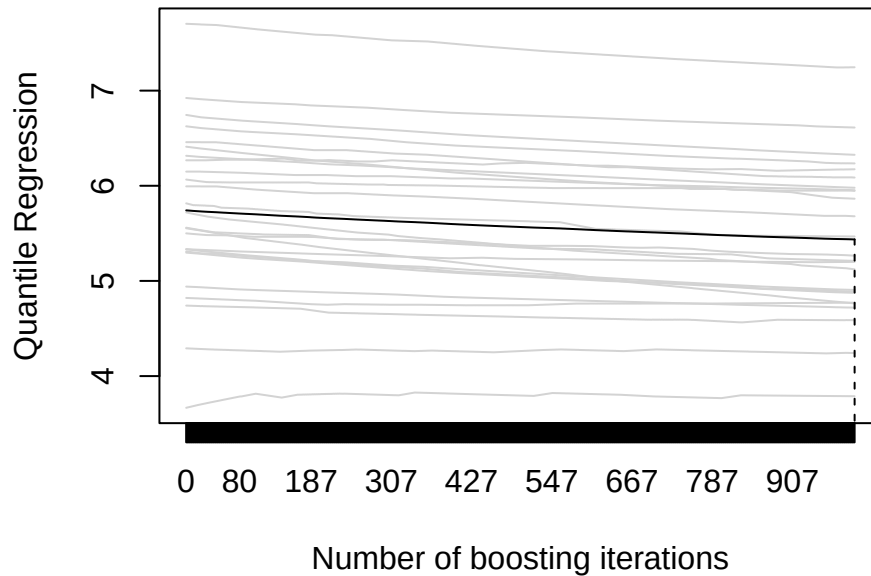
# Predictions:
pred4 <- predict(fit4, newdata=Test)

Acc4 <- round(cor(pred4, Test[,9])^2*100, 2) # Rsquare: Accuracy is 16.14236%
MSE4 <- round(mse(Test[,9], pred4), 2)
RMSE4 <- round(sqrt(mean((Test[,9] - pred4)^2)), 2)
MAE4 <- round(mae(Test[,9], pred4), 2)

set.seed(123)
fit4.cv <- cvrisk(fit4, folds = cv(model.weights(fit4), type = "kfold", B=25)) # B=no of folds

plot(fit4.cv)
```


25-fold kfold



```
#mstop(fit4.cv)
```

We intentionally choose a large maximum number of boosting iterations (`mstop = 1000`). Cross-validation is then used for diagnostic purposes to identify the approximate optimal stopping iteration, but the fitted model is retained at the prespecified number of iterations. The optimal number of boosting iterations corresponds to the value of `mstop` that minimizes the cross-validated risk. Using substantially more iterations may lead to overfitting, whereas too few iterations may result in underfitting. So in practice, we would refit at `mstop = mstop(fit.vc)`.

1.5 Gradient Boosting with Componentwise Smoothing Splines using `gamboost(.)`

Although `gamboost()` allows flexible nonlinear modeling via `bbs()` and linear effects via `bols()`, we intentionally fix the model specification across all methods. In this example, however, we do not include spline base-learners explicitly, so the fitted model remains largely comparable to the previous linear boosting models. Consequently, the results from `gamboost()` are expected to be similar to those obtained from `glmboost()`. Because no nonlinear base learners (`bbs()`) are included, `gamboost` reduces effectively to a linear boosting model. The main advantage of `gamboost()` becomes apparent when nonlinear relationships are incorporated through spline base-learners.

This ensures that differences in predictive performance are attributable only to the estimation algorithm and not to differences in model complexity or functional form.

```
time_model2 <-system.time(model2<- gamboost(fm, data = Train, dfbase = 4,
                                           control = boost_control(mstop = 1000, trace = TRUE)))
```

```
## [ 1] ..... -- risk: 114187.7
```

```
## [ 41] ..... -- risk: 109894.5
## [ 81] ..... -- risk: 109154.8
## [ 121] ..... -- risk: 109018.5
## [ 161] ..... -- risk: 108991.5
## [ 201] ..... -- risk: 108985.7
## [ 241] ..... -- risk: 108984.4
## [ 281] ..... -- risk: 108984.1
## [ 321] ..... -- risk: 108984.1
## [ 361] ..... -- risk: 108984
## [ 401] ..... -- risk: 108984
## [ 441] ..... -- risk: 108984
## [ 481] ..... -- risk: 108984
## [ 521] ..... -- risk: 108984
## [ 561] ..... -- risk: 108984
## [ 601] ..... -- risk: 108984
## [ 641] ..... -- risk: 108984
## [ 681] ..... -- risk: 108984
## [ 721] ..... -- risk: 108984
## [ 761] ..... -- risk: 108984
## [ 801] ..... -- risk: 108984
## [ 841] ..... -- risk: 108984
## [ 881] ..... -- risk: 108984
## [ 921] ..... -- risk: 108984
## [ 961] .....
## Final risk: 108984
```

```
time_5 <- round(as.numeric(time_model2["elapsed"]),2)

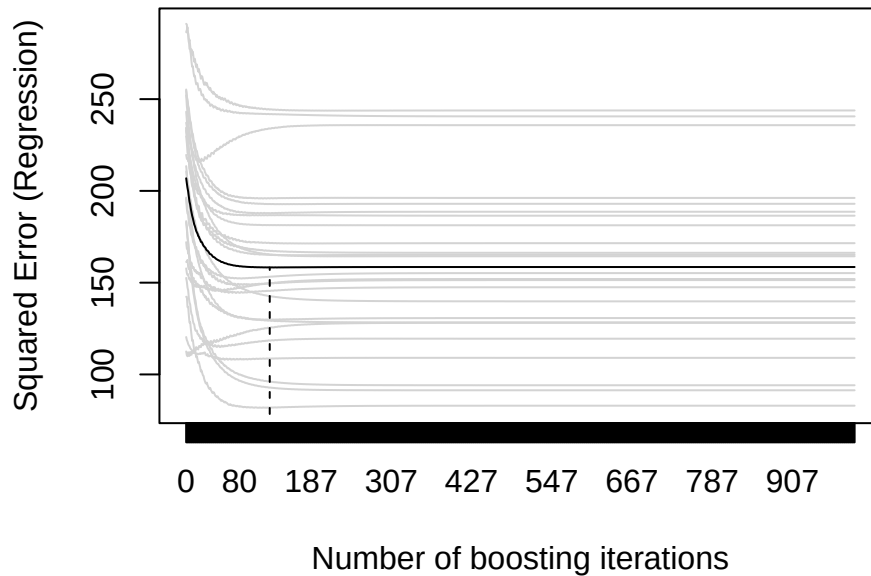
# Predictions:
pred5 <- predict(model2, newdata=Test)
Acc5 <- round(cor(pred5,Test[,9])^2*100, 2) # Rsquare: Accuracy is 19.32%

MSE5 <- round(mse(Test[,9], pred5), 2)
RMSE5 <- round(sqrt(mean((Test[,9] - pred5)^2)), 2)
MAE5 <- round(mae(Test[,9], pred5), 2)

tmp <- capture.output(
  cvm <- cvrisk(
    model2,
    folds = cv(model.weights(model2), type = "kfold", B=25),
    papply = mclapply
  )
) # This suppresses all the "Final risk:" output while still creating the object cvm

#cvm
plot(cvm)
```

25-fold kfold



```
#mstop(cvm) ## extract the optimal mstop
```

The argument `B=25` specifies the number of folds used in cross-validation. Consequently, the data are partitioned into 25 approximately equal-sized subsets, and the plot is automatically labelled as “25-fold k-fold cross-validation”.

1.6 Model-Based Boosting

The model structure is specified via a formula involving the base-learners corresponding to the different model components (linear terms: `bols()`; smooth terms: `bbs()`; interactions: `btrees()`). The model structure (specified using the formula `fm`), the data and the family are then passed to function `mboost()` for model fitting. Again, we keep the model specification fixed (same predictors and additive structure) and vary only the estimation procedure for the same reason stated above.

```
library(mboost)

### fit model
time_model1 <- system.time(model <- mboost(fm, data=Train, family = QuantReg(tau = 0.5),
                                           control = boost_control(mstop = 1000)))#median regression

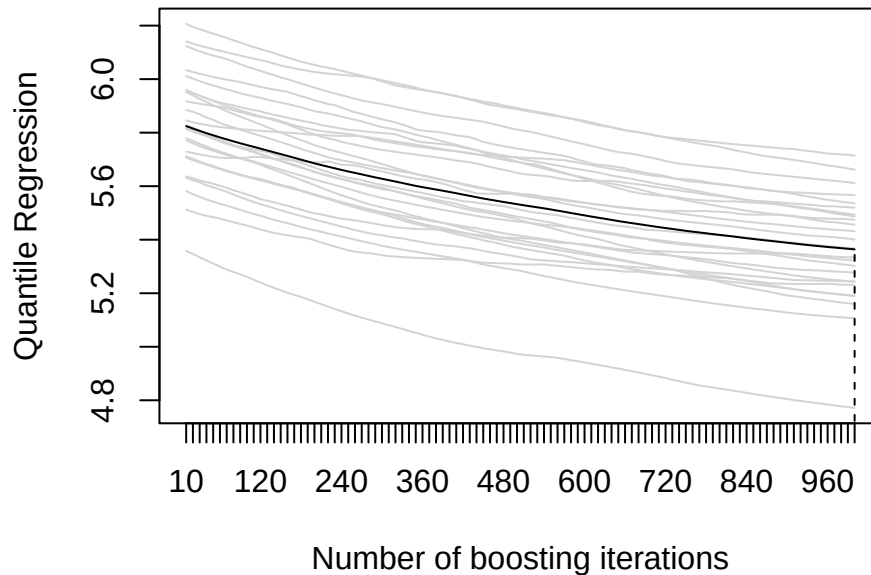
time_6 <- round(as.numeric(time_model1["elapsed"]),2)

# Predictions:
pred6 <- predict(model, newdata=Test)
Acc6 <- round(cor(pred6,Test[,9])^2*100, 2) # Rsquare: Accuracy is 18.56%
MSE6 <- round(mse(Test[,9], pred6) , 2)
RMSE6 <- round(sqrt(mean((Test[,9] - pred6)^2)), 2)
```

```
MAE6 <- round(mae(Test[,9], pred6), 2)

### bootstrap for assessing the 'optimal' number of boosting iterations
cvm <- cvrisk(model, grid = 1:100 * 10)
plot(cvm)
```

25-fold bootstrap



1.7 Boosting (gbm) [for continuous response]

In bagging, bootstrap samples are repeatedly drawn from the training data and the final prediction is obtained by averaging across all trees, thereby reducing variance.

In boosting, models are fitted sequentially, with each new learner attempting to correct the errors made by the previous learners.

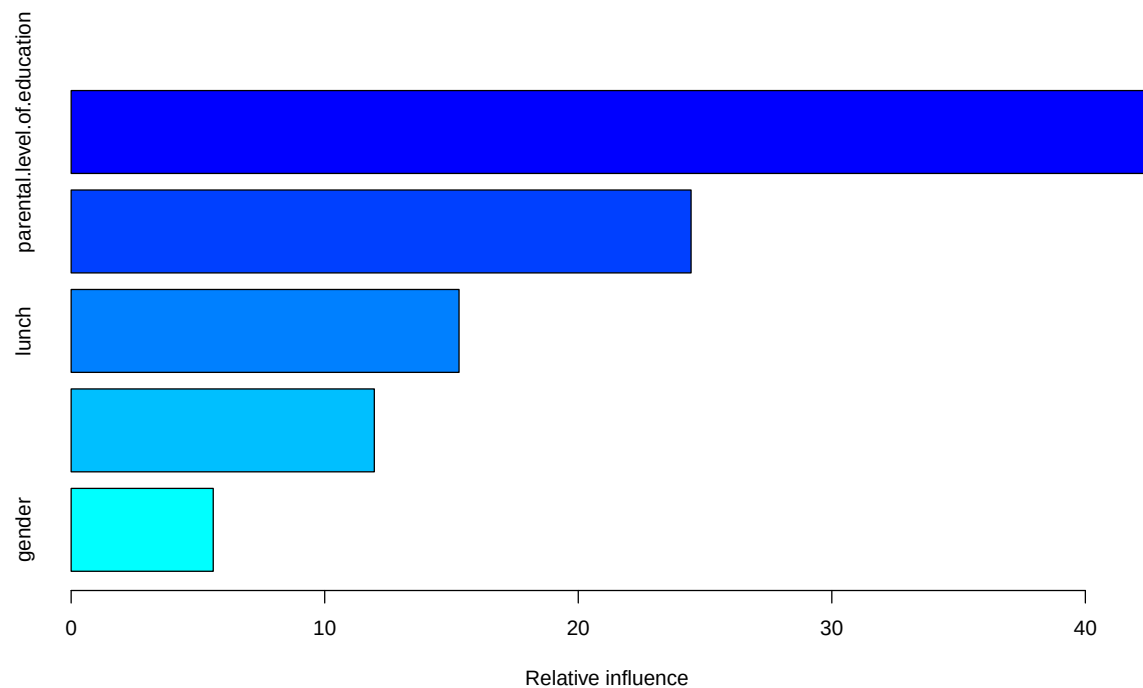
gbm can be fitted to binary, continuous, count and censored data. *gbm* provides multiple risk estimation methods (OOB and CV), which are alternative stopping rules rather than identical procedures.

```
library(gbm)

time_fit_gbm <- system.time(fit_gbm <- gbm(Ave_score ~ gender + test.preparation.course +
  lunch + race.ethnicity + parental.level.of.education ,
  data=Train, distribution="gaussian",
  cv.folds = 5, shrinkage = 0.05, n.trees = 1000))

time_7 <- round(as.numeric(time_fit_gbm["elapsed"]),2)

print(summary(fit_gbm)) # Relative importance of the variables
```



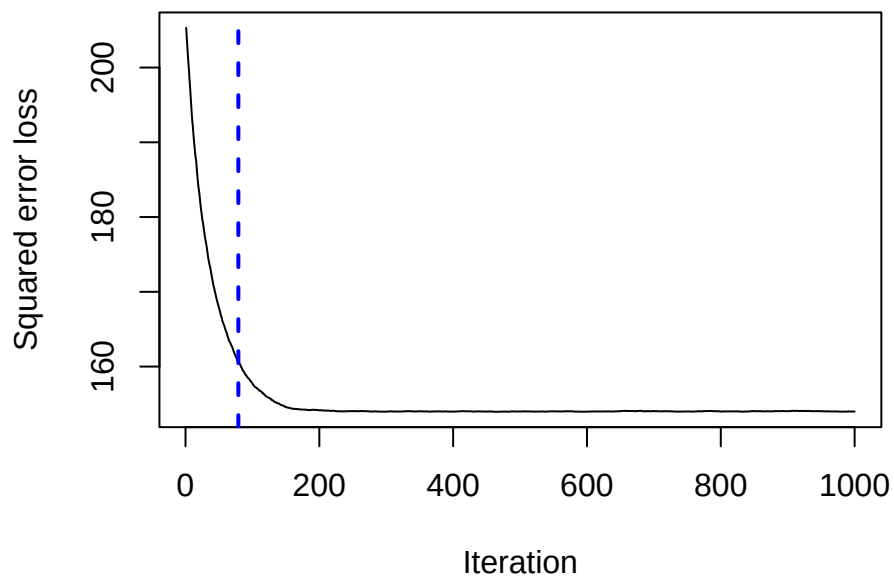
```
##                                var    rel.inf
## parental.level.of.education parental.level.of.education 42.691320
## race.ethnicity                race.ethnicity 24.449430
## lunch                          lunch 15.298441
## test.preparation.course        test.preparation.course 11.957078
## gender                         gender  5.603732
```

```
# Prediction: by default, predictions will be on the link scale
Yhat <- predict(fit_gbm, newdata = Test, n.trees = 1000, type = "response")

Acc7 <- round(cor(Yhat, Test[,9])^2*100, 2) # Rsquare: Accuracy is 19.76%
MSE7 <- round(mse(Test[,9], Yhat), 2)
RMSE7 <- round(sqrt(mean((Test[,9] - Yhat)^2)), 2)
MAE7 <- round(mae(Test[,9], Yhat), 2)
```

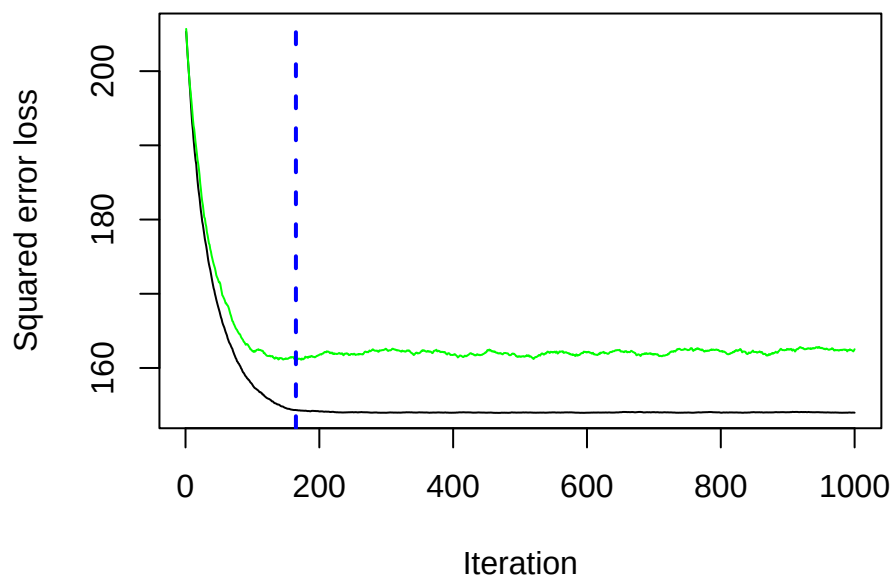
We can check performance using the out-of-bag (OOB) error; although the OOB error typically underestimates the optimal number of iterations.

```
best.iter <- gbm.perf(fit_gbm, method = "OOB")
```



Alternatively, we can check performance using 5-fold cross-validation

```
best.iter2 <- gbm.perf(fit_gbm, method = "cv") # fold is 5 by default
```



Why do they yield different optimal number of iterations?

OOB and cross-validation based estimates differ because:

- OOB estimates are computed from observations that are not used in building a particular tree and therefore provide a computationally efficient estimate of prediction error.
- Cross-validation, on the other hand, repeatedly refits the model on different training folds and generally provides a more stable estimate of the optimal number of iterations.

Unlike standard fixed k-fold cross-validation, the procedure in `mboost` may involve repeated resampling, hence, the folds are not necessarily identical across iterations.

2.0 Binary Boosting Process (Binary classification): Fitting Stochastic Boosting Models

`ada` implements several variants of the AdaBoost algorithm for binary classification, as additive models that can be interpreted through exponential loss minimization and, in later developments, related to additive logistic regression.

Different AdaBoost variants differ mainly in how observation weights are updated, how weak learners are combined, and how the ensemble approximates the underlying loss function (e.g., exponential or logistic interpretations).

Discrete AdaBoost algorithm:

A sequence of weak base learners is fitted, where each learner focuses more strongly on observations that were misclassified in previous iterations. These learners are then combined into a weighted ensemble to form a strong classifier. Misclassified observations receive higher weights in subsequent iterations, while correctly classified observations receive lower weights.

In this application, the response variable `Ave_score` is dichotomized into “Fail” (1–49) and “Pass” (50–100). We explicitly encode the binary response as 0/1 to ensure consistency across models and to make the structure of the loss function explicit, although many implementations can also handle factor labels directly.

Note:

The confusion matrix depends on the ordering of arguments. In `caret::confusionMatrix()`, predictions must be supplied as the first argument and the true labels as the second. Reversing them changes the interpretation of sensitivity, specificity, and predictive values, although overall accuracy remains unchanged.

```
#install.packages("ada")

##### Binary Boosting #####

#####
# The following uses the exponential loss function:
#####

# -- Ada Boost.M1
# -- Real Ada Boost
# -- Gentle Ada Boost

#####
# The following uses logistic loss function:
#####
```

```

# -- Discrete L2 Boost
# -- Real L2 Boost
# -- Gentle L2 Boost

#####

library(rpart)
library(ada)
library(caret)

# Dichotomizing the dataset
Stud = Student_perf
Stud$result <- ifelse(Student_perf$Ave_score < 50, 0, 1)
Stud <- Stud[,c(-6, -7, -8, -9)]
summary(Stud)

```

```

##      gender      race.ethnicity      parental.level.of.education      lunch
## female:518    group A: 89      associate's degree:222      free/reduced:355
## male  :482    group B:190      bachelor's degree :118      standard      :645
##              group C:319      high school      :196
##              group D:262      master's degree  : 59
##              group E:140      some college     :226
##              some high school :179
## test.preparation.course      result
## completed:358      Min.      :0.000
## none      :642      1st Qu.:1.000
##              Median :1.000
##              Mean   :0.897
##              3rd Qu.:1.000
##              Max.   :1.000

```

2.1 Discrete AdaBoost

Uses discrete boosting with an exponential loss function. Ada Boost is often used with simple decision trees as weak base classifiers (significantly improves performance).

```

# data partitioning into Train and Test
set.seed(3000)
n <- dim(Stud)[1]
train <- sample(1:n,.70*n,replace=FALSE)
training <- Stud[train,]
testing <- Stud[-train,]

## fit Ada Boost.M1
time_fit5 <- system.time(fit5 <- ada(result ~ gender + test.preparation.course +
                                   lunch + race.ethnicity + parental.level.of.education,
                                   data=training, loss="exponential",
                                   type="discrete", iter=1000,control=rpart.control())) #no of iter=50 by default

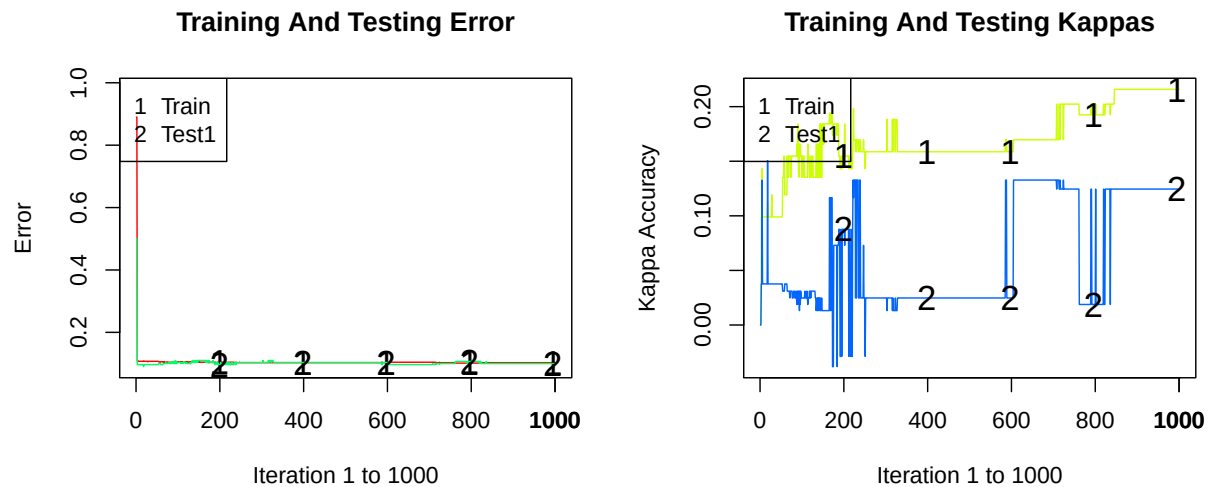
time_8 <- round(as.numeric(time_fit5["elapsed"]),2)
#fit5

```

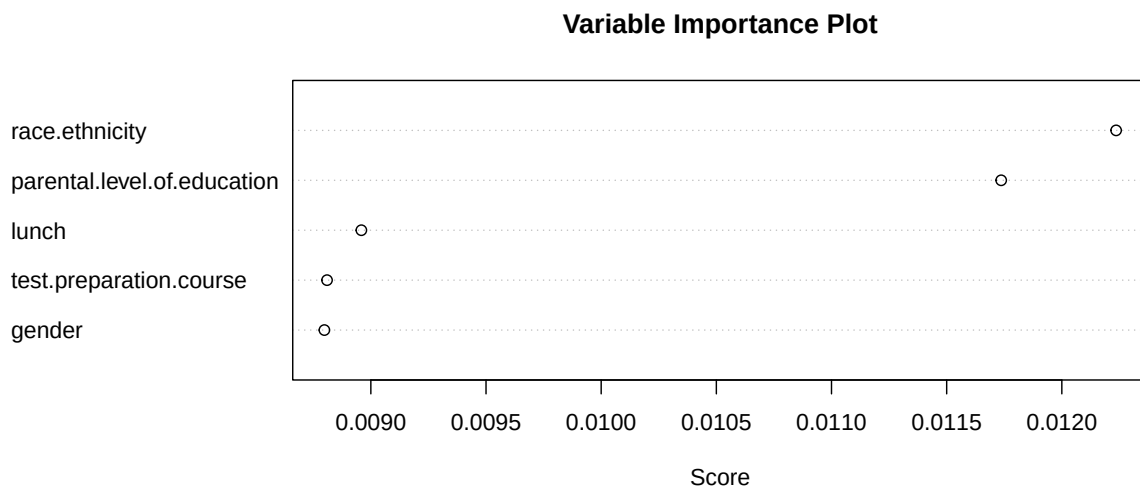


```
## add testing data set
fit6 = addtest(x=fit5, test.x = testing[,-6],test.y = testing[,6])
#summary(fit6)

## plot
plot(fit6,TRUE,TRUE) # Training and testing plot for both (Error and kappas)
```



```
## variable importance plot
varplot(fit6)
```



```
## Prediction
pred.fit6 <- predict(fit6, testing[,-6], "class")# "both", to extract 'Prob' and 'Class'

Ac8 <- confusionMatrix(as.factor(pred.fit6),as.factor(testing[,6]))
Acc8 <- round(Ac8[["overall"]][["Accuracy"]]*100, 2)
```

```
library(verification)

#AUC and ROC
AUC <- roc.area(obs=testing[,6],as.numeric(pred.fit6))$A    #AUC
```

Here, predictions can be obtained either as:

- (i) class labels directly from the model, or
- (ii) probabilities followed by a decision threshold.

We use the direct class prediction to remain consistent with the internal decision rule of the algorithm.

The confusion matrix tells us how well the model was able to classify the testing data. The diagonal elements represent correctly classified observations, whereas the off-diagonal elements represent misclassifications. From the confusion matrix we can compute several performance measures, including accuracy, sensitivity and specificity.

```
confusionMatrix(as.factor(pred.fit6),as.factor(testing[,6]))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0    3    7
##           1   23  267
##
##           Accuracy : 0.9
##           95% CI : (0.8603, 0.9315)
##       No Information Rate : 0.9133
##       P-Value [Acc > NIR] : 0.82341
##
##           Kappa : 0.1245
##
##  Mcnemar's Test P-Value : 0.00617
##
##           Sensitivity : 0.11538
##           Specificity : 0.97445
##       Pos Pred Value : 0.30000
##       Neg Pred Value : 0.92069
##           Prevalence : 0.08667
##       Detection Rate : 0.01000
##       Detection Prevalence : 0.03333
##       Balanced Accuracy : 0.54492
##
##           'Positive' Class : 0
##
```

2.2 Real AdaBoost

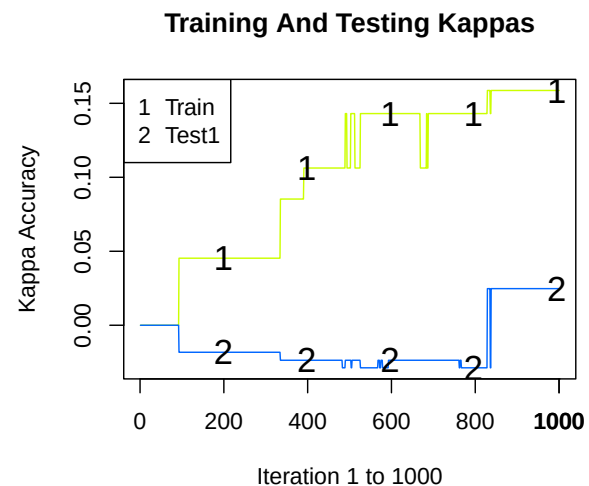
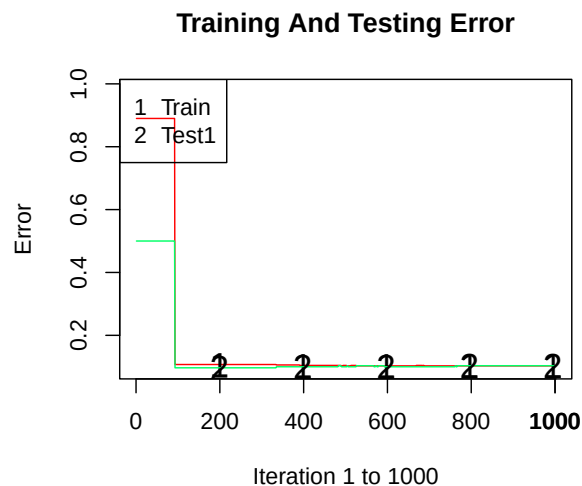
Also uses discrete boosting with an exponential loss function. Only the type of boosting algorithm to perform differs. It is good to note that high number of iterations can lead to over-fitting, while a low number can lead to under-fitting. However, the former appears to be of less risk than the latter.

```
## fit Real Ada Boost
time_fit7 <- system.time(fit7 <- ada(result ~ gender + test.preparation.course +
    lunch + race.ethnicity + parental.level.of.education ,data=training,
    loss="exponential", type="real", iter=1000, control=rpart.control()))

time_9 <- round(as.numeric(time_fit7["elapsed"]),2)
#fit7

## add testing data set to update the model
fit8 = addtest(x=fit7, test.x = testing[,-6],test.y = testing[,6])
#summary(fit8)

## plot
plot(fit8, TRUE, TRUE)
```



```
## Prediction
pred.fit8 <- predict(fit8, testing[,-6], "class")

Ac9 <- confusionMatrix(as.factor(pred.fit8),as.factor(testing[,6]))
Acc9 <- round(Ac9[["overall"]][["Accuracy"]]*100, 2)

# Confusion matrix:
Ac9
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0    1    6
##           1   25  268
##
##           Accuracy : 0.8967
##           95% CI : (0.8565, 0.9287)
##           No Information Rate : 0.9133
##           P-Value [Acc > NIR] : 0.869246
```

```
##
##           Kappa : 0.0247
##
## Mcnemar's Test P-Value : 0.001225
##
##           Sensitivity : 0.038462
##           Specificity : 0.978102
##           Pos Pred Value : 0.142857
##           Neg Pred Value : 0.914676
##           Prevalence : 0.086667
##           Detection Rate : 0.003333
##           Detection Prevalence : 0.023333
##           Balanced Accuracy : 0.508282
##
##           'Positive' Class : 0
##
```

2.3 Gentle AdaBoost

Also uses discrete boosting with an exponential loss function. Algorithm type is “gentle”.

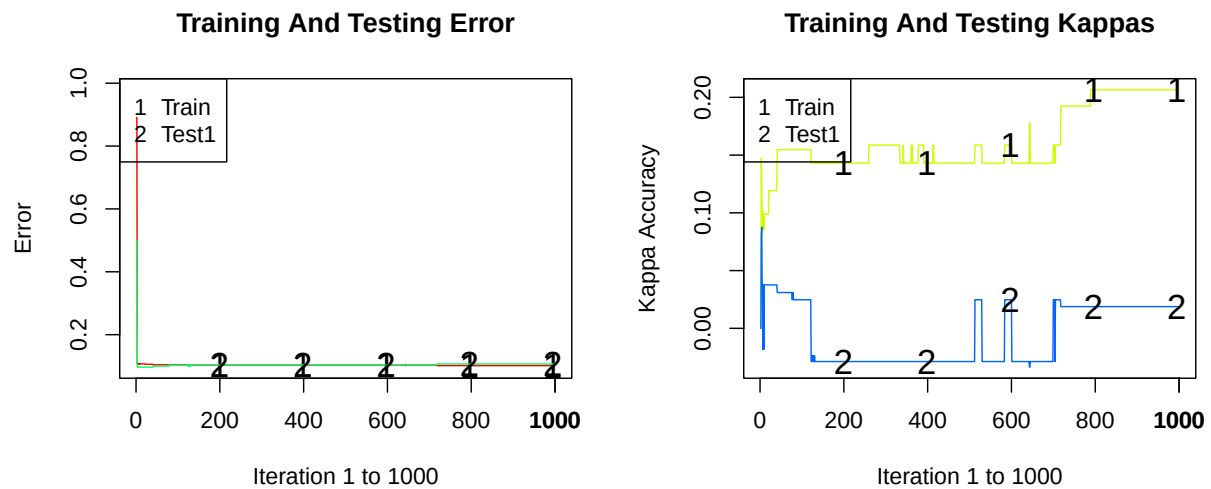
```
## fit Gentle Ada Boost
time_fit9 <- system.time(fit9 <- ada(result ~ gender + test.preparation.course +
                                   lunch + race.ethnicity + parental.level.of.education,
                                   data=training, loss="exponential",
                                   type="gentle",nu=0.1, iter=1000,control=rpart.control()))

time_10 <- round(as.numeric(time_fit9["elapsed"]),2)
#fit9

## add testing data set to update the model
fit10 = addtest(x=fit9, test.x = testing[,-6],test.y = testing[,6])

#summary(fit10)

## plot
plot(fit10, TRUE, TRUE)
```



```
## Prediction
pred.fit10 <- predict(fit10, testing[,-6], "class")

Ac10 <- confusionMatrix(as.factor(pred.fit10),as.factor(testing[,6]))
Acc10 <- round(Ac10[["overall"]][["Accuracy"]]*100, 2)
```

2.4 Discrete L2 Boost

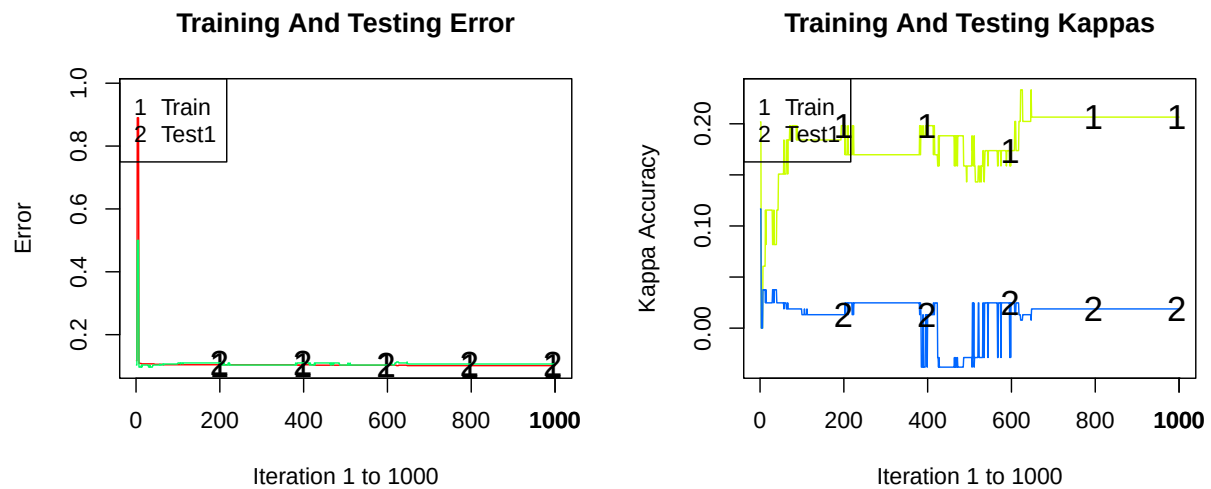
Also uses discrete boosting with a logistic loss function. Algorithm type is “discrete”.

```
## fit Discrete L2 Boost
time_fit11 <- system.time(fit11 <- ada(result ~ gender + test.preparation.course +
  lunch + race.ethnicity + parental.level.of.education,
  data=training, loss="l", type="discrete", nu=0.1,
  iter= 1000, control=rpart.control()))

time_11 <- round(as.numeric(time_fit11["elapsed"]),2)
#fit11

## add testing data set to update the model
fit12 = addtest(x=fit11, test.x = testing[,-6],test.y = testing[,6])
#summary(fit12)

## plot
plot(fit12, TRUE, TRUE)
```



```
## Prediction
pred.fit12 <- predict(fit12, testing[,-6], "class")

Ac11 <- confusionMatrix(as.factor(pred.fit12),as.factor(testing[,6]))
Acc11 <- round(Ac11[["overall"]][["Accuracy"]]*100, 2)
```

2.5 Real L2 Boost

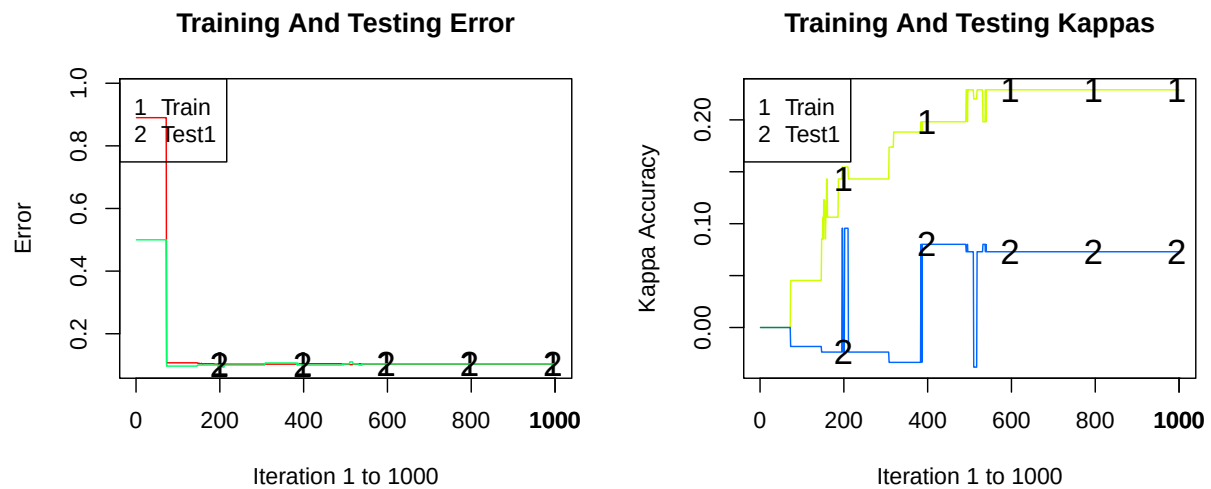
```
## fit Real L2 Boost
time_fit13 <- system.time(fit13 <- ada(result ~ gender + test.preparation.course +
  lunch + race.ethnicity + parental.level.of.education ,
  data=training, loss="l", type="real", nu=0.1,
  iter=1000, control=rpart.control()))

time_12 <- round(as.numeric(time_fit13["elapsed"]),2)
#fit13

## add testing data set to update the model
fit14 = addtest(x=fit13, test.x = testing[,-6],test.y = testing[,6])

#summary(fit14)

## plot
plot(fit14, TRUE, TRUE)
```



```
## Prediction
pred.fit14 <- predict(fit14, testing[,-6], "class")

Ac12 <- confusionMatrix(as.factor(pred.fit14),as.factor(testing[,6]))
Acc12 <- round(Ac12[["overall"]][["Accuracy"]]*100, 2)
```

2.6 Gentle L2 Boost

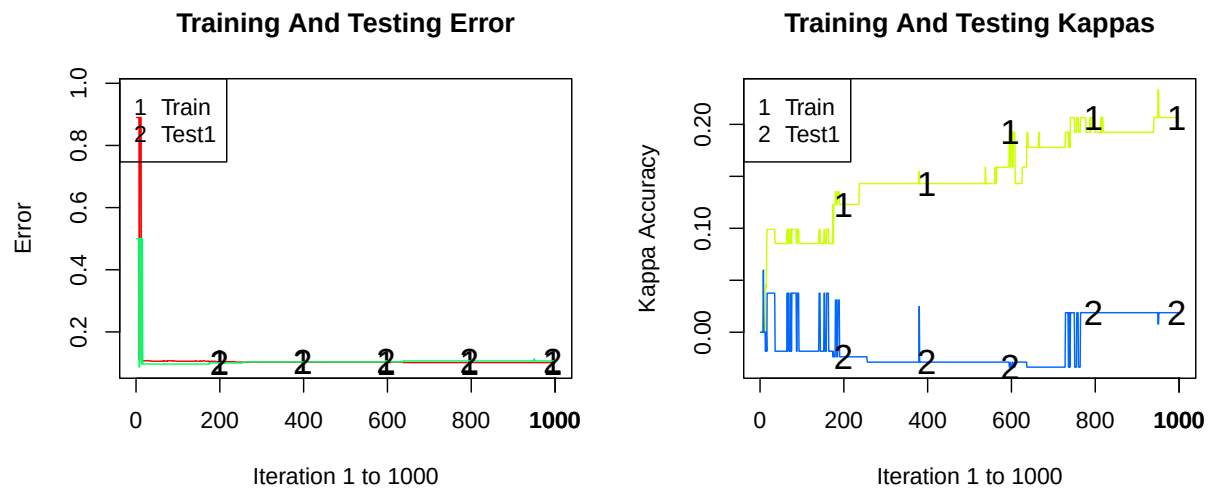
```
## fit Gentle L2 Boost
time_fit15 <- system.time(fit15 <- ada(result ~ gender + test.preparation.course +
  lunch + race.ethnicity + parental.level.of.education ,data=training,
  loss="l", type="gentle", nu=0.1,
  iter = 1000, control=rpart.control())) #no of iter=50 by default

time_13 <- round(as.numeric(time_fit15["elapsed"]),2)
#fit15

## add testing data set to update the model
fit16 = addtest(x=fit15, test.x = testing[,-6],test.y = testing[,6])

#summary(fit16)

## plot
plot(fit16, TRUE, TRUE)
```



```
## Prediction
pred.fit16 <- predict(fit16, testing[,-6], "class")

Ac13 <- confusionMatrix(as.factor(pred.fit16),as.factor(testing[,6]))
Acc13 <- round(Ac13[["overall"]][["Accuracy"]]*100, 2)
```

2.7 eXtreme Gradient Boosting (XGBoost)

Unlike earlier boosting methods implemented in `mboost` or `gbm`, `XGBoost` is an optimized gradient boosting framework designed for computational efficiency, regularization, and scalability to large datasets. It incorporates regularization penalties and several computational optimizations, such as parallel processing and efficient handling of sparse matrices, making it substantially faster than many traditional boosting implementations.

This is an efficient and scalable implementation of gradient boosting framework by (Friedman, 2001). Used for regression, classification and ranking purposes. Note that *XGBoost* requires numeric metrics.

Caution: Tree-based boosting methods, including `XGBoost`, do not naturally extrapolate beyond the range of observed training data in a linear sense. No linear, quadratic, or cubic interpolation is possible. However, `XGBoost` still remains a very attractive tool for bringing out structure in complex data with many features. Using it for forecasting time series can be a good win, as long your target is stationary. Fits response with 2 or more categories.

```
##### XGBoost for timeseries with Python
```

```
library(xgboost)
library(magrittr)
library(dplyr)
library(Matrix)

# For a classification problem

## requires Binary response starting from zero
Stud1 <- Stud
Stud1[,6] <- as.numeric(Stud1[,6])
```



```

##set up testing and training data (70% for training)
set.seed(3000)
n <- dim(Stud1)[1]
train <- sample(1:n,.70*n,replace=FALSE)
trainy <- Stud1[train,]
testy <- Stud1[-train,]

# Create matrix (will also do a One-Hot Encoding for Factor variables if it exist)
trainm <- sparse.model.matrix(result ~. -1, data = trainy) # -1 to exclude intercept in the matrix

train_label <- trainy[, "result"]
train_matrix <- xgb.DMatrix(data = as.matrix(trainm), label = train_label)

testm <- sparse.model.matrix(result ~. -1, data = testy)
test_label <- testy[, "result"]
test_matrix <- xgb.DMatrix(data = as.matrix(testm), label = test_label)

nc <- length(unique(train_label))
xgb_params <- list("objective" = "multi:softprob",
                  "eval_metric" = "mlogloss",
                  "num_class" = nc)
watchlist <- list(train = train_matrix, test = test_matrix)

# eXtreme Gradient Boosting Model
# first a simple model
#####
set.seed(123)
bst_model <- xgb.train(params = xgb_params,
                      data = train_matrix,
                      nrounds = 100, # max number of iterations
                      watchlist = watchlist,
                      eta = 0.01, # low value = robust to overfitting
                      max.depth = 3, #default value is 6
                      gamma = 0, #larger values=more conservative algorithm
                      subsample = 0.5, #betw 0-1, lower values=to prevent overfitting
                      colsample_bytree = 1, #default value is 1
                      missing = NA, #useful when we have alot of missing data
                      seed = 333,
                      verbose = 0)

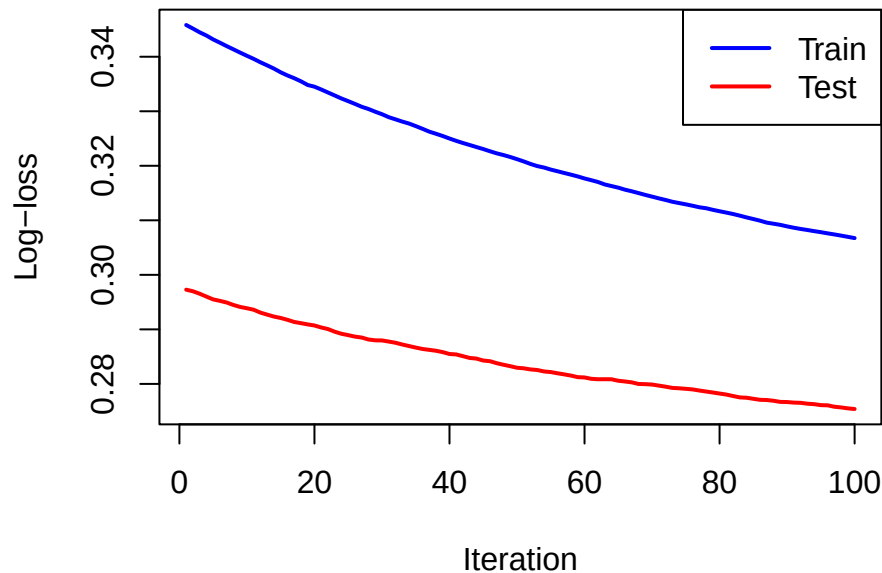
# Training & test error plot
#####
e <- data.frame(attributes(bst_model)$evaluation_log)

plot(e$iter, e$train_mlogloss,
     type = "l", col = "blue", lwd = 2,
     ylim = range(c(e$train_mlogloss, e$test_mlogloss)),
     xlab = "Iteration", ylab = "Log-loss")

lines(e$iter, e$test_mlogloss, col = "red", lwd = 2)

```

```
legend("topright", legend = c("Train", "Test"),
      col = c("blue", "red"), lwd = 2)
```



```
e$test_mlogloss[which.min(e$test_mlogloss)] # minimum loss\error in testdata
```

```
## [1] 0.2753975
```

```
e$iter[which.min(e$test_mlogloss)] # iter at minimum loss in testdata
```

```
## [1] 100
```

Instead of choosing number iterations blindly, we can introduce *early stopping*. This will allow the model to automatically stop near the best iteration.

Note that `early_stopping_rounds = 20` does not mean that training stops at iteration 20. Instead, it means stop training only if after the evaluation metric fails to improve for 20 consecutive iterations.

In summary, early stopping selects the model iteration with the best evaluation metric on the validation set and restores that model automatically.

```
## Now let us update the model
#####
set.seed(123)
time_bst_model2 <- system.time(bst_model2 <- xgb.train(params = xgb_params,
  data = train_matrix,
  nrounds = 1000, # now updated
  watchlist = watchlist,
```

```

        early_stopping_rounds = 20, # stop if no improvement
        eta = 0.01,
        max.depth = 3,
        gamma = 0,
        subsample = 0.5,
        colsample_bytree = 1,
        missing = NA,
        seed = 333,
        verbose = 0))

time_14 <- round(as.numeric(time_bst_model2["elapsed"]),2)

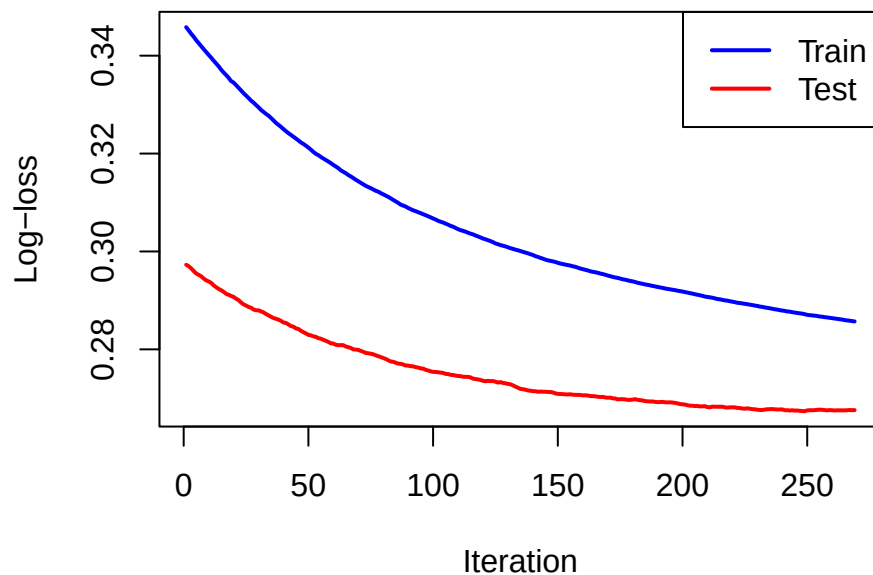
e2 <- data.frame(attributes(bst_model2)$evaluation_log)

plot(e2$iter, e2$train_mlogloss,
     type = "l", col = "blue", lwd = 2,
     ylim = range(c(e2$train_mlogloss, e2$test_mlogloss)),
     xlab = "Iteration", ylab = "Log-loss")

lines(e2$iter, e2$test_mlogloss, col = "red", lwd = 2)

legend("topright", legend = c("Train", "Test"),
      col = c("blue", "red"), lwd = 2)

```



```

e2$test_mlogloss[which.min(e2$test_mlogloss)] # minimum loss\error in testdata

```

```
## [1] 0.2673661
```

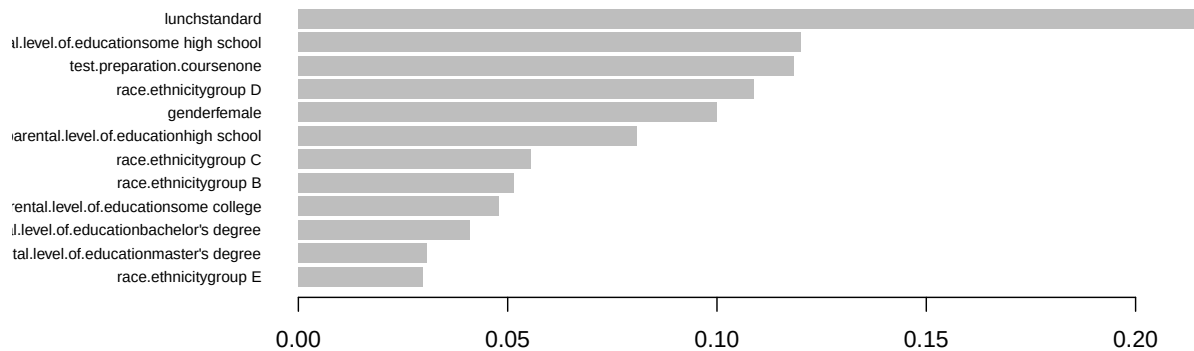
```
e2$iter[which.min(e2$test_mlogloss)] # iter at minimum loss in testdata
```

```
## [1] 249
```

For feature importance:

```
# Feature importance
```

```
imp <- xgb.importance(colnames(train_matrix), model = bst_model2)
xgb.plot.importance(imp)
```



```
# Prediction & confusion matrix - test data
```

```
pred.fit17 <- predict(bst_model2, newdata = test_matrix)
```

```
pred_class <- max.col(matrix(pred.fit17, ncol = nc, byrow = TRUE)) - 1
```

```
Ac14 <- confusionMatrix(as.factor(pred_class), as.factor(testing[,6]))
```

```
Acc14 <- round(Ac14[["overall"]][["Accuracy"]]*100, 2)
```

For the prediction, since XGBoost returns class probabilities for each category, the final class is obtained by selecting the category with the highest predicted probability.

2.8 gbm() with binary response

```
library(caret)
```

```
library(gbm)
```

```
#####
#### Method 1 (with bernoulli distribution) and train() function ####
#####
```

```

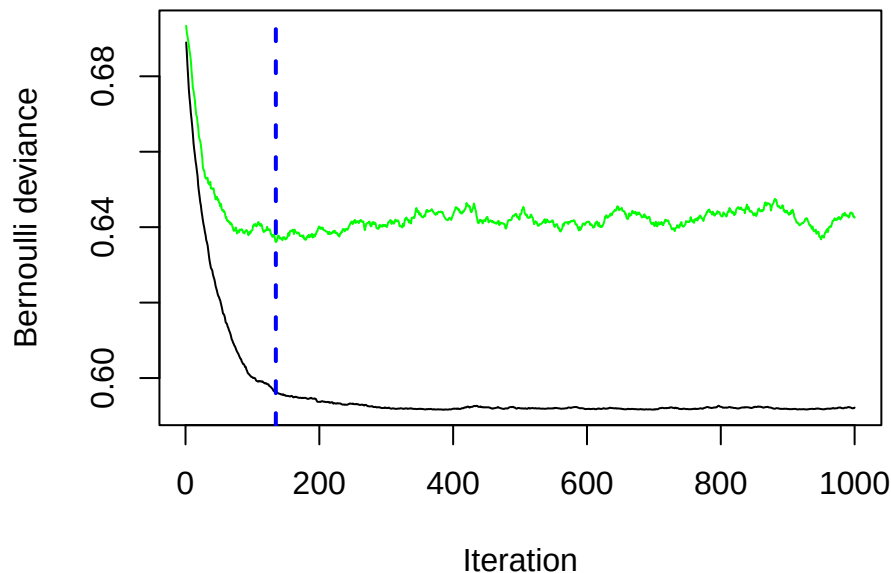
set.seed(123)
time_fitgbm <- system.time(fit.gbm <- gbm(result ~ gender + test.preparation.course +
    lunch + race.ethnicity + parental.level.of.education, data=trainy,
    distribution="bernoulli", cv.folds = 5, shrinkage = 0.05, n.trees = 1000))

time_15 <- round(as.numeric(time_fitgbm["elapsed"]), 2)

#summary(fit.gbm)

# Check performance using 5-fold cross-validation:
# Using cv_folds>1 when calling gbm usually results in improved predictive performance
best.iter <- gbm.perf(fit.gbm, method = "cv")

```



```

# Prediction: by default, predictions will be on the response scale
Yhat <- predict(fit.gbm, newdata = testy, n.trees=best.iter, na.action=na.pass, type="response")
Yhat <- round(Yhat)

```

Since the Bernoulli model returns estimated probabilities of belonging to class 1, we convert the probabilities into class labels using a threshold of 0.5.

```

##### Model performance checking

Ac15 <- confusionMatrix(as.factor(Yhat), as.factor(testy[,6]))
Acc15 <- round(Ac15[["overall"]][["Accuracy"]]*100, 2)

#AUC
AUC15 <- roc.area(Yhat, testy[,6])$A

```

```
#####
#### Method 2 (with adaboost distribution) ####
#####

set.seed(123)
fitControl = trainControl(method="cv", number=5, returnResamp = "all")

gbmfit = train(as.factor(result) ~., data=training, method="gbm",distribution="adaboost",
               trControl=fitControl, verbose=F,
               tuneGrid=data.frame(.n.trees=best.iter,
                                   .shrinkage=0.05,
                                   .interaction.depth=1,
                                   .n.minobsinnode=1))

#gbmfit

# Model performance checking
confusionMatrix(gbmfit) # percentual average cell counts across resamples

## Cross-Validated (5 fold) Confusion Matrix
##
## (entries are percentual average cell counts across resamples)
##
##           Reference
## Prediction    0    1
##           0  0.1  0.0
##           1 10.9 89.0
##
## Accuracy (average) : 0.8914

# Predictions
mPred = predict(gbmfit, testing, n.trees=best.iter, na.action = na.pass)

getTrainPerf(gbmfit)

## TrainAccuracy TrainKappa method
## 1      0.8914365 0.02262443      gbm

postResample(mPred, as.factor(testing[,6]))

## Accuracy      Kappa
## 0.9133333 0.0000000

# confusion matrix for the test data
confusionMatrix(mPred,as.factor(testing[,6]))

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
```

```
##          0    0    0
##          1   26 274
##
##          Accuracy : 0.9133
##          95% CI : (0.8756, 0.9426)
##    No Information Rate : 0.9133
##    P-Value [Acc > NIR] : 0.552
##
##          Kappa : 0
##
##    McNemar's Test P-Value : 9.443e-07
##
##          Sensitivity : 0.00000
##          Specificity : 1.00000
##    Pos Pred Value :      NaN
##    Neg Pred Value : 0.91333
##          Prevalence : 0.08667
##    Detection Rate : 0.00000
##    Detection Prevalence : 0.00000
##    Balanced Accuracy : 0.50000
##
##    'Positive' Class : 0
##
```

3.0 Boosting for Multicategory Classification

In multicategory classification, the response variable takes more than two unordered labels (here: poor, good, excellent). Most boosting algorithms first produce a score or probability per class, and class labels are obtained by taking the maximum probability across classes. Evaluation is therefore based on classification accuracy and confusion matrices rather than regression-style error measures.

```
# -- SAMME
# -- Breiman's Extension
# -- Freund's Adjustment
```

3.1 SAMME

SAMME is an extension of the AdaBoost algorithm to multiclass problems. Instead of being limited to two classes, it can handle multiple categories by modifying the weight update rule. The key idea is that SAMME increases the weight of misclassified observations so that subsequent weak learners focus more on difficult cases, while also scaling each learner's contribution based on the number of classes. The parameter α_m depends on the classification error and the number of classes, ensuring that weak learners contribute appropriately even in multiclass settings.

Intuitively, SAMME repeatedly builds weak classifiers that focus more on observations that were previously misclassified.

```
#install.packages("adabag")
library(adabag)
library(caTools)

# Trichotomizing the dataset
```

```

Stud2 = Student_perf
Stud2$grade<- rep(NA, nrow(Student_perf))
Stud2[Stud2$Ave_score < 55, ][, "grade"] <- "poor"
Stud2[Stud2$Ave_score >= 55 & Stud2$Ave_score < 80, ][, "grade"] <- "good"
Stud2[Stud2$Ave_score >= 80, ][, "grade"] <- "Excellent"
Stud2$grade <- as.factor(Stud2$grade)
Stud2 <- Stud2[ ,c(-6, -7, -8, -9)]
summary(Stud2)

```

```

##      gender      race.ethnicity      parental.level.of.education      lunch
## female:518    group A: 89      associate's degree:222      free/reduced:355
## male  :482    group B:190      bachelor's degree :118      standard      :645
##                                     group C:319      high school      :196
##                                     group D:262      master's degree   : 59
##                                     group E:140      some college     :226
##                                     some high school :179
## test.preparation.course      grade
## completed:358      Excellent:198
## none      :642      good      :611
##                                     poor      :191
##
##
##

```

```

set.seed(3001)
spl = sample.split(Stud2$grade, SplitRatio = 0.7)
Traindata = subset(Stud2, spl==TRUE)
Testdata = subset(Stud2, spl==FALSE)

formula = grade ~.
time_fit17 <- system.time(fit17 <- boosting(formula, data=Traindata, boos = TRUE,
                                           mfina1 = 500, coeflearn = 'Zhu', control = rpart.control()))

time_16 <- round(as.numeric(time_fit17["elapsed"]),2)

#for coeflearn = 'Zhu', SAMME algorithm is implemented with alpha=ln((1-err)/err)+ln(nclasses-1).

table(fit17$class, Traindata[,6]) # confusion matrix with respect to Train data

```

```

##
##      Excellent good poor
## Excellent      40  24   2
## good           90 376  71
## poor           9  28  61

```

```

# v-fold cross validation with AdaBoost.M1 or SAMME:
#####
# The data are divided into v non-overlapping subsets of roughly equal size.
# Then, boosting is applied on (v-1) of the subsets. Finally, predictions are
# made for the left out subsets, and the process is repeated for each of the v subsets

fit17.cv <- boosting.cv(formula, data=Traindata, v=10, mfina1=100,
                        control = rpart.control(), coeflearn = "Zhu")

```



```
## i:  1 Wed Jul  1 13:17:51 2026
## i:  2 Wed Jul  1 13:17:55 2026
## i:  3 Wed Jul  1 13:17:58 2026
## i:  4 Wed Jul  1 13:18:02 2026
## i:  5 Wed Jul  1 13:18:05 2026
## i:  6 Wed Jul  1 13:18:09 2026
## i:  7 Wed Jul  1 13:18:12 2026
## i:  8 Wed Jul  1 13:18:16 2026
## i:  9 Wed Jul  1 13:18:19 2026
## i: 10 Wed Jul  1 13:18:23 2026
```

```
#fit17.cv$class
fit17.cv$confusion
```

```
##              Observed Class
## Predicted Class Excellent good poor
##      Excellent          23   71    9
##      good            105  316   93
##      poor             11   41   32
```

```
#fit17.cv$error
```

```
# Prediction
```

```
pred_boosting1 <- predict.boosting(fit17, newdata = Testdata)
pred_boosting1$confusion
```

```
##              Observed Class
## Predicted Class Excellent good poor
##      Excellent          9   18    0
##      good            48  142   38
##      poor             2   23   19
```

```
#pred_boosting1$error
```

```
Ac16 <- confusionMatrix(as.factor(pred_boosting1$class), as.factor(Testdata[,6]))
Acc16 <- round(Ac16[["overall"]][["Accuracy"]]*100, 2)
```

```
#AUC
```

```
library(pROC)
AUC16 <- as.numeric(multiclass.roc(as.numeric(as.factor(pred_boosting1$class)),
                                   as.numeric(Testdata[,6]))$auc)
AUC16 <- round(AUC16,2)
```

3.2 Breiman's Extension

Breiman's version of AdaBoost modifies the weighting scheme by scaling the contribution of each weak learner differently from SAMME. While SAMME is explicitly derived as a multiclass extension of AdaBoost with a principled adjustment for the number of classes, Breiman's original formulation was developed for binary classification. In practice, it can be extended to multiclass settings in a heuristic way, but unlike SAMME, it is not directly derived from a multiclass-specific exponential loss function.

```
# if 'Breiman'(by default),  $\alpha=1/2*\ln((1-err)/err)$  is used

time_fit18 <- system.time(fit18 <- boosting(formula, data=Traindata, boos = TRUE, mfinal = 500,
      coeflearn = 'Breiman', control = rpart.control()))

time_17 <- round(as.numeric(time_fit18["elapsed"]),2)

table(fit18$class, Traindata[,6]) # confusion matrix with respect to Train data
```

```
##
##           Excellent good poor
## Excellent      12    5    1
## good          126   414  112
## poor           1    9   21
```

```
#fit6$importance
sort(fit18$importance, decreasing = TRUE)
```

```
## parental.level.of.education      race.ethnicity
##           34.481825                26.278979
##           lunch      test.preparation.course
##           18.726122                14.623989
##           gender
##           5.889084
```

```
fit18.cv <- boosting.cv(formula, data=Traindata, v=10, mfinal=100,
      control = rpart.control(), coeflearn = "Breiman")
```

```
## i:  1 Wed Jul  1 13:18:44 2026
## i:  2 Wed Jul  1 13:18:48 2026
## i:  3 Wed Jul  1 13:18:51 2026
## i:  4 Wed Jul  1 13:18:55 2026
## i:  5 Wed Jul  1 13:18:58 2026
## i:  6 Wed Jul  1 13:19:02 2026
## i:  7 Wed Jul  1 13:19:05 2026
## i:  8 Wed Jul  1 13:19:09 2026
## i:  9 Wed Jul  1 13:19:12 2026
## i: 10 Wed Jul  1 13:19:16 2026
```

```
#fit18.cv$class
fit18.cv$confusion
```

```
##           Observed Class
## Predicted Class Excellent good poor
##           Excellent      7    8    1
##           good          131   408  123
##           poor           1    12   10
```

```
fit18.cv$error # misclassification rate
```

```
## [1] 0.3937233
```

```
# Prediction
pred_boosting2 <- predict.boosting(fit18, newdata = Testdata)
pred_boosting2$confusion
```

```
##              Observed Class
## Predicted Class Excellent good poor
##      Excellent          2    5    0
##      good             57   172   51
##      poor              0    6    6
```

```
pred_boosting2$error
```

```
## [1] 0.3979933
```

```
Ac17 <- confusionMatrix(as.factor(pred_boosting2$class), as.factor(Testdata[,6]))
Acc17 <- round(Ac17[["overall"]][["Accuracy"]]*100, 2)

AUC17 <- as.numeric(multiclass.roc(as.numeric(as.factor(pred_boosting2$class)),
                                   as.numeric(Testdata[,6]))$auc)
AUC17 <- round(AUC17,2)
```

3.3 Freund's Adjustment

Freund's variant typically places stronger emphasis on misclassified observations, which can lead to improved training accuracy but also increases the risk of overfitting.

Among the three variants, Freund's method is typically the most aggressive in focusing on misclassified observations.

```
time_fit19 <- system.time(fit19 <- boosting(formula, data=Traindata, boos = TRUE, mfinal = 500,
                                           coeflearn = 'Freund', control = rpart.control()))

time_18 <- round(as.numeric(time_fit19["elapsed"]),2)

table(fit19$class, Traindata[,6]) # confusion matrix with respect to Train data
```

```
##
##              Excellent good poor
##      Excellent          12    5    1
##      good             126   416  115
##      poor              1    7   18
```

```
# variable importance
#sort(fit19$importance, decreasing = TRUE)
```

```
fit19.cv <- boosting.cv(formula, data=Traindata, v=10, mfinal=100,
                        control = rpart.control(), coeflearn = "Freund")
```

```
## i: 1 Wed Jul 1 13:19:38 2026
## i: 2 Wed Jul 1 13:19:41 2026
## i: 3 Wed Jul 1 13:19:45 2026
## i: 4 Wed Jul 1 13:19:48 2026
## i: 5 Wed Jul 1 13:19:51 2026
## i: 6 Wed Jul 1 13:19:55 2026
## i: 7 Wed Jul 1 13:19:58 2026
## i: 8 Wed Jul 1 13:20:02 2026
## i: 9 Wed Jul 1 13:20:05 2026
## i: 10 Wed Jul 1 13:20:09 2026
```

```
#fit19.cv$class
fit19.cv$confusion
```

```
##              Observed Class
## Predicted Class Excellent good poor
##      Excellent          5    7    1
##      good           133  407  121
##      poor            1   14   12
```

```
#fit19.cv$error # misclassification rate
```

```
# Prediction
```

```
pred_boosting3 <- predict.boosting(fit19, newdata = Testdata)
pred_boosting3$confusion #pred_boosting3$error #
```

```
##              Observed Class
## Predicted Class Excellent good poor
##      Excellent          1    4    0
##      good           58  172  53
##      poor            0    7    4
```

```
Ac18 <- confusionMatrix(as.factor(pred_boosting3$class), as.factor(Testdata[,6]))
Acc18 <- round(Ac18[["overall"]][["Accuracy"]]*100, 2)
```

```
AUC18 <- as.numeric(multiclass.roc(as.numeric(as.factor(pred_boosting3$class)),
                                   as.numeric(Testdata[,6]))$auc)
```

```
AUC18 <- round(AUC18,2)
```

3.4 using gbm()

Gradient Boosting Machines (GBM) build models sequentially by optimizing a loss function using gradient descent. Each tree is fitted to the negative gradient of the loss function, which corresponds to residuals in the case of squared error loss.

```
tc = trainControl(method = "repeatedcv", number = 5)
gbmGrid <- expand.grid(interaction.depth = c(1, 3, 5),
                      n.trees = 1000, # (1:25)*25,
                      shrinkage = 0.1,
                      n.minobsinnode = 10)
```

```

set.seed(444) # for reproducibility
time_model <- system.time(model3 <- train(grade ~., data=Traindata, method="gbm",
                                         trControl=tc, verbose = FALSE, tuneGrid = gbmGrid))

time_19 <- round(as.numeric(time_model["elapsed"]),2)
#model

# predict test data.

pred = predict(model3, Testdata)
result = data.frame(Testdata$grade, pred)

# print(result)

Ac19 <- confusionMatrix(as.factor(pred), as.factor(Testdata$grade))
Acc19 <- round(Ac19[["overall"]][["Accuracy"]]*100, 2)

#AUC
AUC19 <- as.numeric(multiclass.roc(as.numeric(as.factor(pred)), as.numeric(Testdata[,6]))$auc)
AUC19 <- round(AUC19,2)

```

3.5 using xgboost()

XGBoost is an advanced implementation of gradient boosting that includes regularization to reduce overfitting. It improves computational efficiency and model accuracy by optimizing both the loss function and model complexity. Unlike classical boosting methods, XGBoost includes built-in mechanisms such as shrinkage, column sampling, and tree pruning.

Before now XGBoost requires the classes to be in an integer format, starting with 0. Presently, the new `xgboost()` interface infers the task from `y`, and a numeric `y` is treated as regression while a factor `y` will be regarded as meant for classification problem.

In multiclass problems, XGBoost estimates a probability distribution over all classes for each observation, and the final class is selected based on the highest probability.

Compared to GBM, XGBoost is faster, more regularized, and generally more stable in high-dimensional settings.

```

library(xgboost)
train_x = data.matrix(Traindata[,-6])
train_y = Traindata[,6]

test_x = data.matrix(Testdata[,-6])
test_y = Testdata[,6]

# the model
time_xgboost <- system.time(
  xgbc <- xgboost(
    data = train_x,
    label = train_y,
    #objective = "multi:softprob", # no longer mandatory in the new interface
    num_class = 3,
    max_depth = 5,

```

```

    nrounds = 500,
    eval_metric = "mlogloss"
  )
)
time_20 <- round(as.numeric(time_xgboost["elapsed"]),2)

# Predicting test data
pred <- predict(xgbc, newdata = test_x)

# Predicted class names
pred_class <- colnames(pred)[max.col(pred, ties.method = "first")]
# ties.method = "first" means If there is a tie, pick the first column.

# Convert to a factor
pred_class <- factor(pred_class, levels = levels(Testdata$grade))

# check the prediction accuracy with a confusion matrix.
cm = confusionMatrix(pred_class, test_y)
print(cm)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  Excellent good poor
##  Excellent      11    25    1
##    good         45   133   38
##    poor          3    25   18
##
## Overall Statistics
##
##              Accuracy : 0.5418
##              95% CI : (0.4835, 0.5993)
##    No Information Rate : 0.612
##    P-Value [Acc > NIR] : 0.99432
##
##              Kappa : 0.0911
##
##  McNemar's Test P-Value : 0.02445
##
## Statistics by Class:
##
##              Class: Excellent Class: good Class: poor
## Sensitivity           0.18644      0.7268      0.3158
## Specificity           0.89167      0.2845      0.8843
## Pos Pred Value        0.29730      0.6157      0.3913
## Neg Pred Value        0.81679      0.3976      0.8458
## Prevalence            0.19732      0.6120      0.1906
## Detection Rate        0.03679      0.4448      0.0602
## Detection Prevalence  0.12375      0.7224      0.1538
## Balanced Accuracy      0.53905      0.5056      0.6000

```

```

Ac20 <- confusionMatrix(pred_class, test_y)
Acc20 <- round(Ac20[["overall"]][["Accuracy"]]*100, 2)

```

```
AUC20 <- as.numeric(multiclass.roc(as.numeric(pred_class),
                                   as.numeric(test_y))$auc)
AUC20 <- round(AUC20,2)
```

Bonus model : Neural network (Multiclass Classification)

Neural networks provide an alternative approach to multiclass classification by learning a flexible nonlinear mapping between predictors and class probabilities.

Unlike tree-based boosting methods, neural networks do not build an additive sequence of weak learners. Instead, they estimate a single model with internal hidden layers, trained jointly using backpropagation to minimize classification loss, that can capture complex interactions between variables.

```
library(nnet)
time_nnet <- system.time(mn.net <- nnet::multinom(grade ~., Traindata, maxit = 500))
```

```
## # weights: 42 (26 variable)
## initial value 770.127214
## iter 10 value 589.143524
## iter 20 value 583.648796
## iter 30 value 583.604074
## iter 30 value 583.604073
## iter 30 value 583.604073
## final value 583.604073
## converged
```

```
time_21 <- round(as.numeric(time_nnet["elapsed"]),2)

# Use predict with type="prob" to get class probabilities
nnet_predict <- predict(mn.net, newdata=Testdata, type="prob")
#head(nnet_predict)

# This can be used directly in multiclass.roc:
AUC21 <- as.numeric(multiclass.roc(Testdata$grade, nnet_predict)$auc)
AUC21 <- round(AUC21,2)
```

Results

```
# create table for the 5 models
table = matrix(NA, nrow = 21, ncol = 8)

# create column names
colnames(table) = c("Type", "Model", "Runtime", "Accuracy", "RMSE", "MAE", "AUC", "mstep")

## Stick in the needed information
library(Metrics)

AUC8 <- round(roc.area(obs=testing[,6], as.numeric(pred.fit6))$A, 2)
```

```

AUC9 <- round(roc.area(obs=testing[,6],as.numeric(pred.fit8))$A, 2)
AUC10 <- round(roc.area(obs=testing[,6],as.numeric(pred.fit10))$A, 2)
AUC11 <- round(roc.area(obs=testing[,6],as.numeric(pred.fit12))$A, 2)
AUC12 <- round(roc.area(obs=testing[,6],as.numeric(pred.fit14))$A, 2)
AUC13 <- round(roc.area(obs=testing[,6],as.numeric(pred.fit16))$A, 2)
AUC14 <- round(roc.area(obs=testing[,6],as.numeric(pred_class))$A, 2)
AUC15 <- round(roc.area(Yhat, testy[,6])$A, 2)

table[1, ] = c("Continuous" , "L2Reg", time_1 , "-", RMSE1, MAE1, "" , 1000)
table[2, ] = c("", "L1Reg", time_2 , "-", RMSE2, MAE2, "" , 1000)
table[3, ] = c("", "RobustReg", time_3 , "-", RMSE3, MAE3, "" , 1000)
table[4, ] = c("", "QuantReg", time_4 , "-", RMSE4, MAE4, "" , 1000)
table[5, ] = c("", "gamboost", time_5 , "-", RMSE5, MAE5, "" , 1000)
table[6, ] = c("", "mboost", time_6 , "-", RMSE6, MAE6, "" , 1000)
table[7, ] = c("", "gbm", time_7 , "-", RMSE7, MAE7, "" , 1000)
table[8, ] = c("Binary" , "ada(dis)", time_8 , Acc8, "-", "-", AUC8, 1000)
table[9, ] = c("", "ada(Real)", time_9 , Acc9, "-", "-", AUC9, 1000)
table[10, ] = c("", "ada(Gentle)", time_10 , Acc10, "-", "-", AUC10, 1000)
table[11, ] = c("", "ada(dis L2)", time_11 , Acc11, "-", "-", AUC11, 1000)
table[12, ] = c("", "ada(Real L2)", time_12 , Acc12, "-", "-", AUC12, 1000)
table[13, ] = c("", "ada(Gentle L2)", time_13 , Acc13, "-", "-", AUC13, 1000)
table[14, ] = c("", "Xgboost", time_14 , Acc14, "-", "-", AUC14, 1000)
table[15, ] = c("", "gbm", time_15 , Acc15, "-", "-", AUC15, 1000)
table[16, ] = c("Multicategory" , "boosting(SAMME)", time_16 , Acc16, "-", "-", AUC16, 500)
table[17, ] = c("", "boosting(Breiman's)", time_17 , Acc17, "-", "-", AUC17, 500)
table[18, ] = c("", "boosting(Freund's)", time_18 , Acc18, "-", "-", AUC18, 500)
table[19, ] = c("", "gbm", time_19 , Acc19, "-", "-", AUC19, 500)
table[20, ] = c("", "Xgboost", time_20 , Acc20, "-", "-", AUC20, 500)
table[21, ] = c("", "Neural network", time_21 , Acc20, "-", "-", AUC21, 500)

# print table

library(knitr)
kable(table, caption = "**Comparisons**")

```

Table 1: Comparisons

Type	Model	Runtime	Accuracy	RMSE	MAE	AUC	mstep
Continuous	L2Reg	0.07	-	12.5	10.11		1000
	L1Reg	0.16	-	12.97	10.44		1000
	RobustReg	0.25	-	12.5	10.13		1000
	QuantReg	0.08	-	13.29	10.67		1000
	gamboost	0.3	-	12.5	10.1		1000
	mboost	0.29	-	12.74	10.27		1000
	gbm	3.02	-	12.5	10.11		1000
Binary	ada(dis)	9.29	90	-	-	0.54	1000
	ada(Real)	9.22	89.67	-	-	0.51	1000

Type	Model	Runtime	Accuracy	RMSE	MAE	AUC	mstep
Multicategory	ada(Gentle)	8.7	89.33	-	-	0.51	1000
	ada(dis L2)	9.9	89.33	-	-	0.51	1000
	ada(Real L2)	10.28	89.67	-	-	0.53	1000
	ada(Gentle L2)	9.28	89.33	-	-	0.51	1000
	Xgboost	1.36	47.33	-	-	0.5	1000
	gbm	3.5	90.33	-	-	0.46	1000
	boosting(SAMME)	16.91	56.86	-	-	0.69	500
	boosting(Breiman's)	16.72	60.2	-	-	0.71	500
	boosting(Freund's)	17.31	59.2	-	-	0.66	500
	gbm	14.59	59.87	-	-	0.74	500
	Xgboost	4.58	54.18	-	-	0.66	500
	Neural network	0.03	54.18	-	-	0.67	500